



AI-агенты в разработке: системный подход

Кто я

Работаю в компании вебпрактик

Бэкэнд разработчик

Тимлид

90%

Структура доклада

Что такое AI агенты и как они работают

Выбор инструментов и моделей

Подготовка проекта и контекста

Постановка задачи

Что такое AI агенты и как они работают

Структура раздела

- Классификация AI инструментов
- Что такое агенты
- Как агенты меняют процесс разработки

Эволюция AI инструментов в разработке

Чат → Автокомплит → Агенты → Бэкграунд агенты

Киллер-фичи агентов

Видит содержимое папки, в которой запущен

Может запускать CLI команды

Буквально пишет код вместо тебя

Агенты != silver bullet

Требуют хорошей инженерной культуры

Подготовки проекта

За код и результат ВСЕГДА отвечает человек, а не агент

Если агенты не прижились у вас в команде - возможно проблема не в агентах



Разработка с AI агентами != Вайбкодинг

Вайбкодинг - пишешь промпты под музыку и лавандовый раф, надеешься что нейронка сгенерирует именно то, о чём ты думаешь

AI агенты - мощный инструмент. Но с большой силой приходит большая ответственность

AI агенты - новый подход к написанию кода

- Мы перестаём быть механическими писателями кода.
- Основная задача — качественное формирование требований и ревью результата

Ты не только разработчик, но и тимлид

И ты тоже

И я

И каждый, у кого есть команда из AI агентов

Задачи те же

Представь, что к тебе приходит новый джун

Он хорошо умеет писать код, но ничего не знает о твоём проекте

Без онбординга, понятных инструкций и ревью может (и будет) чудить

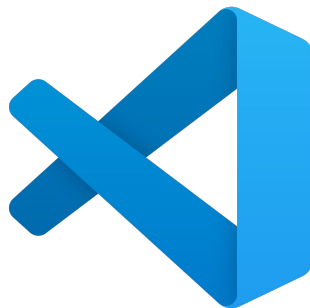
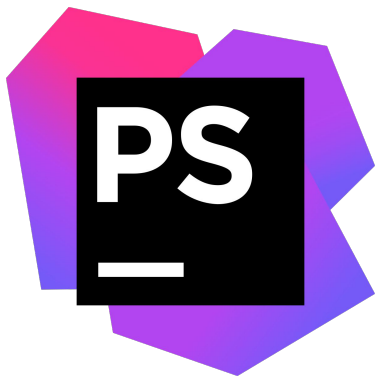
Выбор инструментов и моделей

Структура раздела

- IDE и редакторы кода
- Плагины
- CLI
- Выбор моделей



IDE и редакторы кода



Cursor

- С него начался хайп
- Слабо адаптирован под PHP
- Быстро заканчиваются лимиты
- Агенты - кор функционал

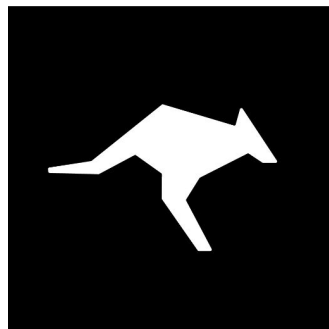


PhpStorm

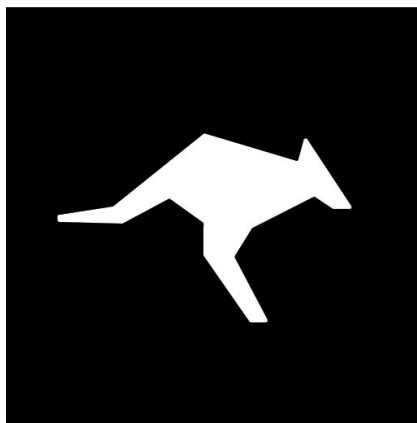
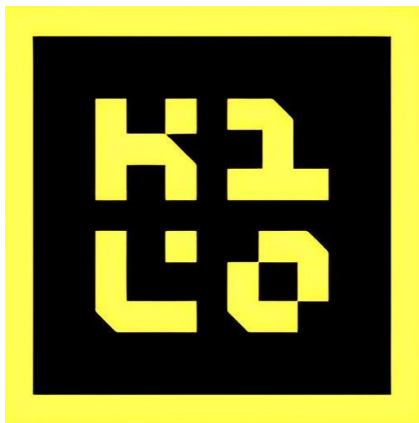
- Опоздал на несколько месяцев
- Отлично адаптирован под PHP
- **ОЧЕНЬ** быстро заканчиваются лимиты
- Агенты - вспомогательный функционал



Плагины



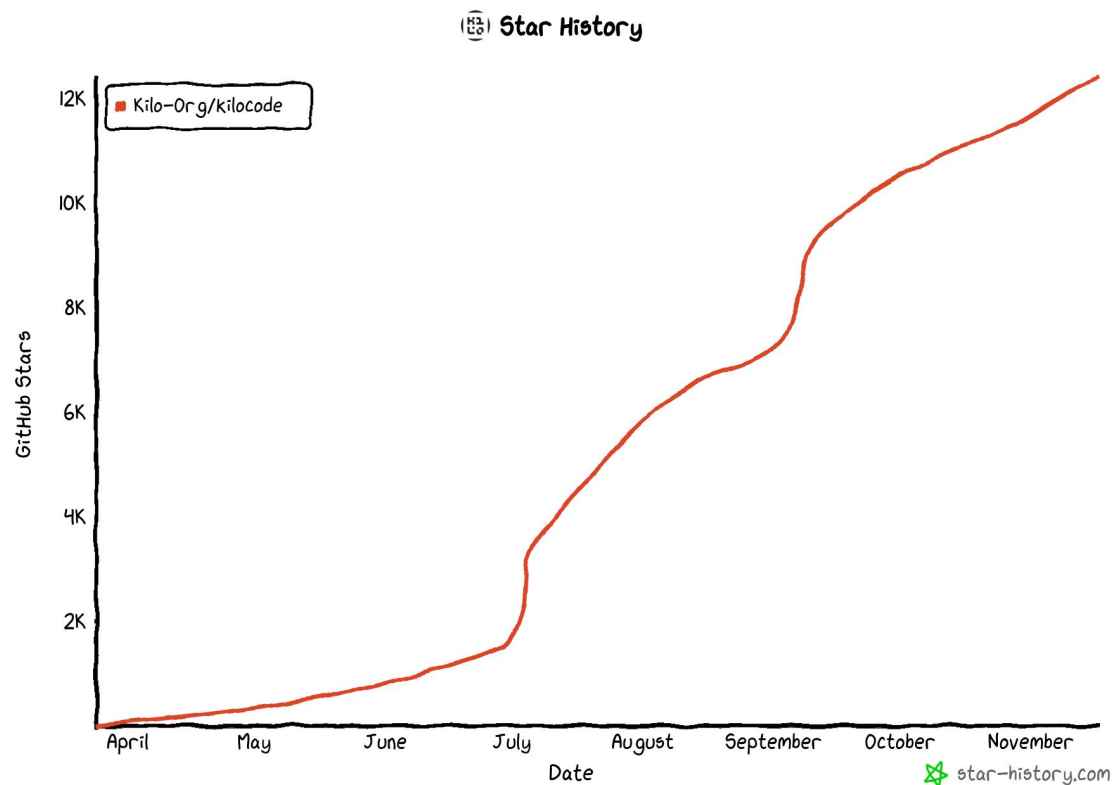
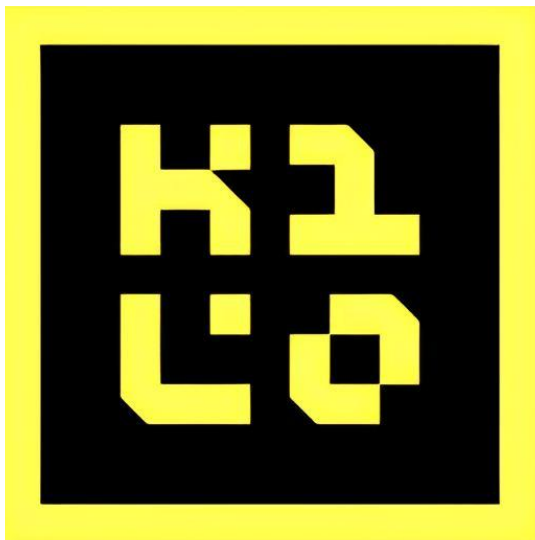
Было у отца три сына



В семье ни без windsurf'а

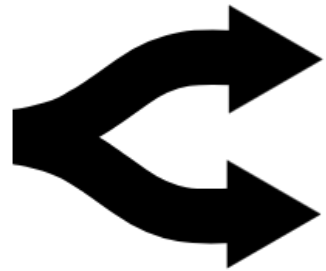


Kilo Code



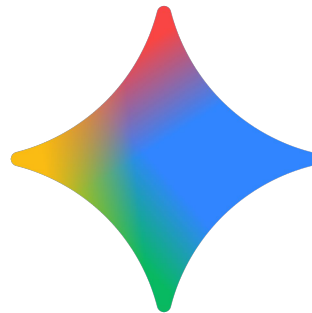
Kilo Code: Ключевые преимущества

- Стабильная работа в PHPStorm
- Встроенный Memory Bank
- Широкий выбор AI провайдеров
- Агенты с ролями



OpenRouter

CLI



Claude Code

- Передовые модели
- Лучший UX среди CLI
- Хорошее соотношение
лимитов и стоимости
- Активное развитие, новые фичи

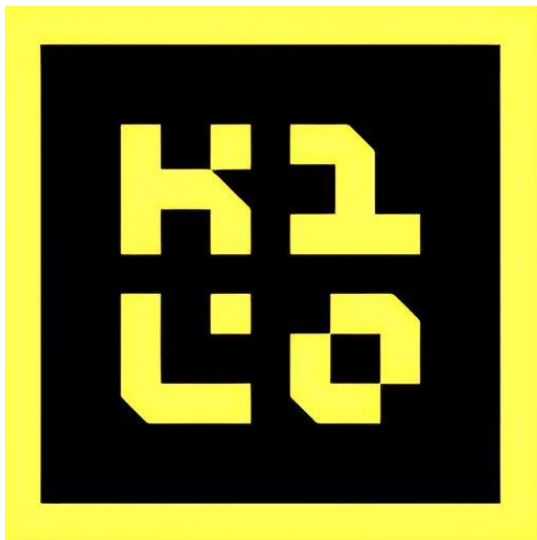


Qwen Code

- Бесплатно и доступно в РФ
- Неплохие модели
- Хороший UX
- Активное развитие, новые фичи



Мои рекомендации

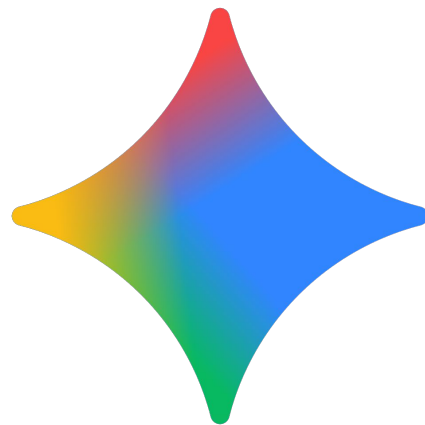


Выбор модели

Выбор модели: SWE bench

Место	Модель	Процент
1	Gemini 3 Pro	71.6
2	GPT 5.1 Codex	70.4
3	Claude Sonnet 4.5 Thinking	69.8
4	GPT 5 Codex	69.4
5	Claude Haiku 4.5 Thinking	68.8

Выбор модели



Выбор модели

Проприетарные модели:

- Claude sonnet 4.5
- GPT-5.1 Codex
- Gemini 3 Pro

Выбор модели

Опенсорсные модели

- GLM 4.6
- MiniMax 2
- Qwen 3 coder
- Kimi k2
- DeepSeek V3.1

Мои рекомендации



Подготовка проекта и контекста

Структура раздела

- [Agents.md](#)
- Memory bank
- MCP
- Безопасность

Я не в контексте

И агент тоже. Нужно дать ему весь необходимый контекст

- Контекст вашей команды\компании
- Контекст проекта

Важно - агент помнит НАМНОГО меньше контекста, чем человек. Можно дообучить агента под свой проект и\или команду, но такое доступно только избранным

Agents.md

Это база, это знать надо

Файл с правилами для агента, которые он добавляет в контекст при каждом запуске

Чтоб создать первую итерацию чаще всего достаточно запустить команду `init`

[Agents.md](#) - не константа

Agents.md

Базовый минимум:

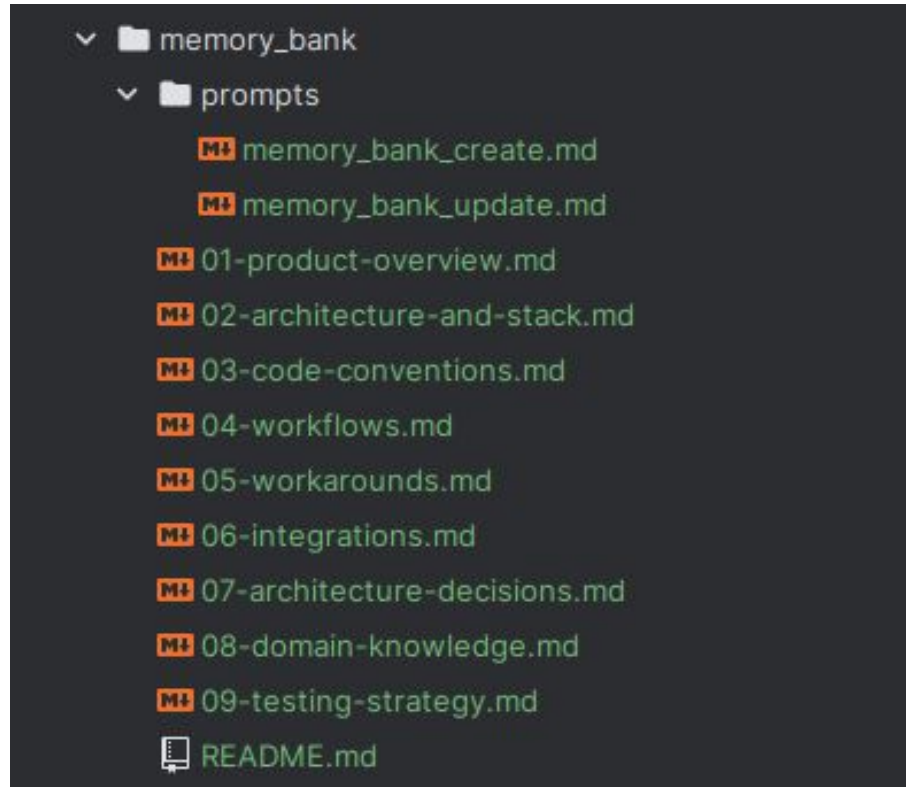
- Основные правила работы агента
- Краткое описание проекта и стека
- Кодстайл и прочие стандарты

Даже такая базовая конфигурация даст неплохой буст в плане предсказуемости результата и снизит количество галлюцинаций

Memory bank

- Следующий шаг в документировании проекта
- Хорошей практикой будет сразу подготовить промпт для актуализации документации
- Добавьте информацию о memory bank в agents.md
- Если ваша база знаний лежит отдельно от кода - не беда, используйте MCP

Memory bank



А на русском писать можно?

- Можно! А еще лучше - на польском!



← Новость

Исследование →



MCP

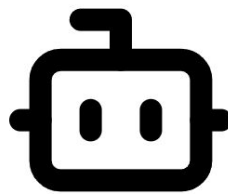
Важный шаг в подготовке проекта - подключение нужных инструментов

На практике MCP — волшебный портал во внешний мир для агента. Можно дать доступ к таск-трекеру и документации, приложениям, актуальной документации

MCP



Context 7

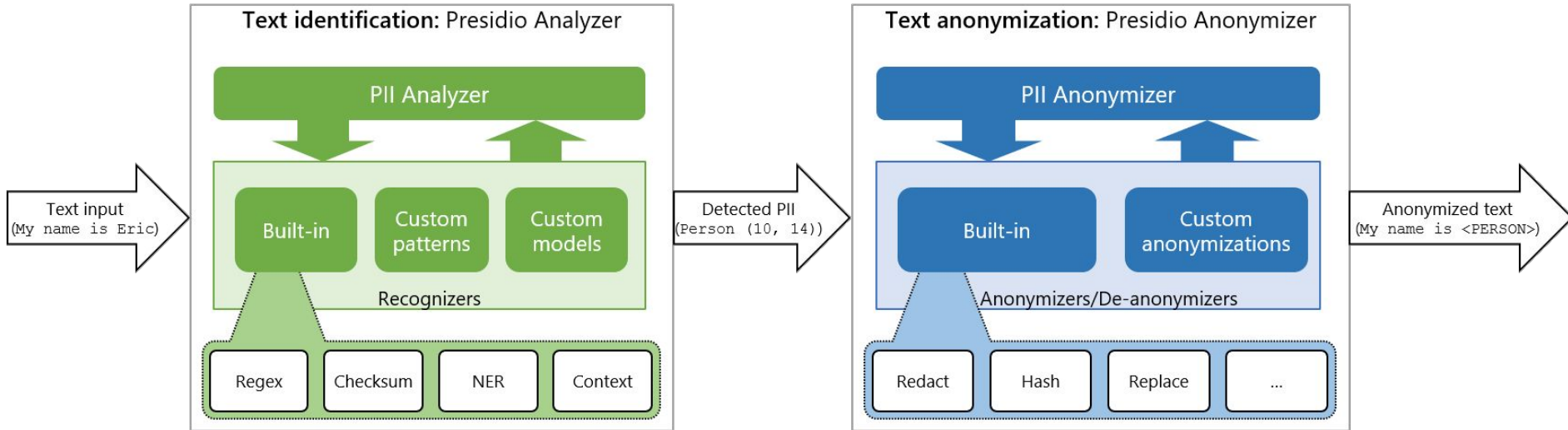


Browser MCP

Безопасность на уровне конфигов: .agentignore

Запрети агентам читать энвы, порадуй безопасника хоть немного

Proxy - на примере Microsoft Presidio



Постановка задачи

Я не в контексте. Опять

Агент теперь в курсе о проекте и ваших стандартах

Но без понятия что хочет клиент, что было на созвоне и что дизайнер нарисовал в фигме

Как НЕ надо ставить задачу агенту

“Сделай красиво!”

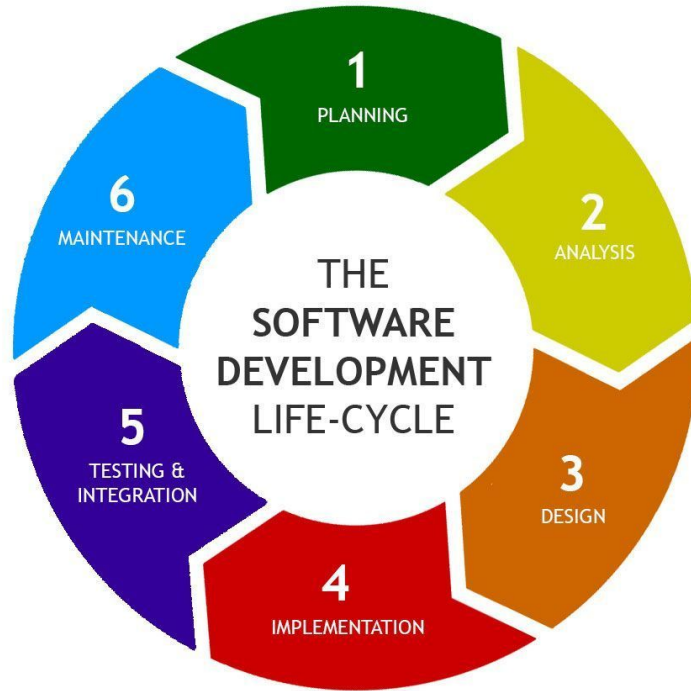
После этого промпта в коде начался сущий кошмар

Вот задача - сделай ее

Это может работать для старта новых проектов, для прототипирования

Не работает на средних и тем более больших проектах

Жизненный цикл задачи для агента



Жизненный цикл задачи для агента

1. Анализируем задачу и проект

Жизненный цикл задачи для агента

1. Анализируем задачу и проект
2. Разбиваем на небольшие подзадачи (планирование)

Жизненный цикл задачи для агента

1. Анализируем задачу и проект
2. Разбиваем на небольшие подзадачи (планирование)
3. Прорабатываем план реализации. Тут же появляется спецификация, заготовка документации - в общем много md файлов

Жизненный цикл задачи для агента

1. Анализируем задачу и проект
2. Разбиваем на небольшие подзадачи (планирование)
3. Прорабатываем план реализации. Тут же появляется спецификация, заготовка документации - в общем много md файлов
4. Проводим ревью плана

Жизненный цикл задачи для агента

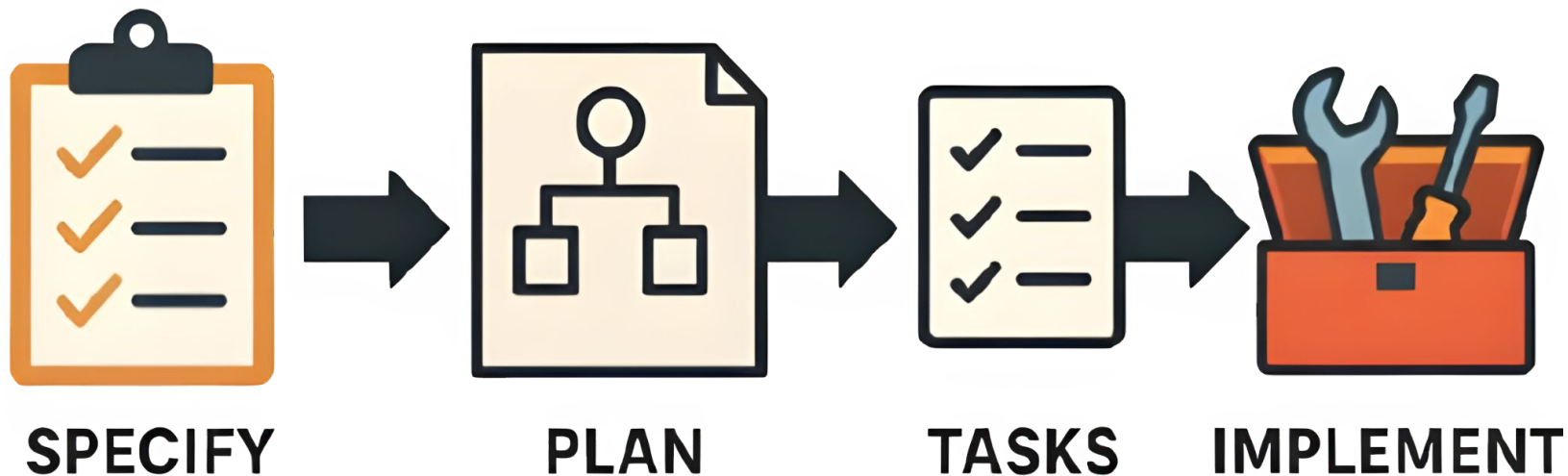
1. Анализируем задачу и проект
2. Разбиваем на небольшие подзадачи (планирование)
3. Прорабатываем план реализации. Тут же появляется спецификация, заготовка документации - в общем много md файлов
4. Проводим ревью плана
5. Пишем код

Жизненный цикл задачи для агента

1. Анализируем задачу и проект
2. Разбиваем на небольшие подзадачи (планирование)
3. Прорабатываем план реализации. Тут же появляется спецификация, заготовка документации - в общем много md файлов
4. Проводим ревью плана
5. Пишем код
6. Проводим ревью результата

**Агента можно и нужно использовать на всех этапах,
а не ТОЛЬКО для написания кода**

Spec-Driven Development



TODO поменять картинку

Spec-Driven Development

- GitHub Spec Kit
- OpenSpec
- Kiro
- Qoder

Выводы

Вместо выводов - Gartner Hype Cycle



Скил работы с AI - это скилл разработчика

Спасибо за внимание!

Бенчмарки

<https://lmarena.ai/leaderboard/webdev>

<https://lmarena.ai/leaderboard>

<https://swe-rebench.com/>

https://swe-rebench.com/?insight=oct_2025

<https://www.vals.ai/benchmarks/swebench>

Выбор модели:

Место	Модель	Баллы
1	Gemini 3 Pro	1487
2	GPT 5 Medium	1395
3	Claude Opus 4.1	1394
4	Claude Sonnet 4.5 Thinking	1392
5	Claude Sonnet 4.5	1387

Цель

Рассмотреть агентов как инструмент для разработки. Дать инструкции и рекомендации, которые можно применить на практике для продуктивного использования AI агентов

Если после моего доклада вы решите дать агентам шанс или прокачаете свою работу с агентам - цель достигнута

Бонус: Пример на практике

По ходу презентации мы пройдем полный путь - выбор инструментов, подготовка проекта, постановка и выполнение задачи

Todo:

Показать пример настроек для своего проекта

Показать пример проработки задачи

Залить несколько полезных файлов-примеров в гит - обезличенные конфиги для проекта, пример исходной задачи и промптов и тд

WebDev Leaderboard

Compare the performance of AI models for web development tasks built in the Code Arena
The legacy WebDev leaderboard is still available at web.lmarena.ai

Last Updated

Nov 12, 2025

Total Votes

3,217

Total Models

11

Search by model name...

Rank ↕	Rank Spread ⓘ (Upper-Lower)	Model ↕	Score ↓ ⓘ	95% CI (±) ↕	Votes ↕	Organization ↕	License ↕
1	1 ↔ 5	 claude-opus-4-1-20250805	1404	+25/-25	579	Anthropic	Proprietary
2	1 ↔ 5	 claude-sonnet-4-5-20250929-thinking-32k	1401	+24/-24	593	Anthropic	Proprietary
3	1 ↔ 5	 gpt-5-medium	1387	+25/-25	624	OpenAI	Proprietary
4	1 ↔ 5	 claude-sonnet-4-5-20250929	1387	+23/-23	759	Anthropic	Proprietary
5	1 ↔ 5	 glm-4.6	1373	+26/-26	573	Z.ai	MIT
6	6 ↔ 8	 qwen3-coder-480b-a35b-instruct	1278	+24/-24	576	Alibaba	Apache 2.0
7	6 ↔ 8	 minimax-m2	1263	+24/-24	581	MiniMax	Apache 2.0
8	6 ↔ 8	 claude-haiku-4-5-20251001	1255	+23/-23	640	Anthropic	Proprietary
9	9 ↔ 10	 gemini-2.5-pro	1174	+25/-25	592	Google	Proprietary
10	9 ↔ 11	 grok-code-fast-1	1137	+54/-54	132	xAI	Proprietary

Arena Overview

Scroll to the right to see full stats of each model 

First Place
Second Place
Third Place

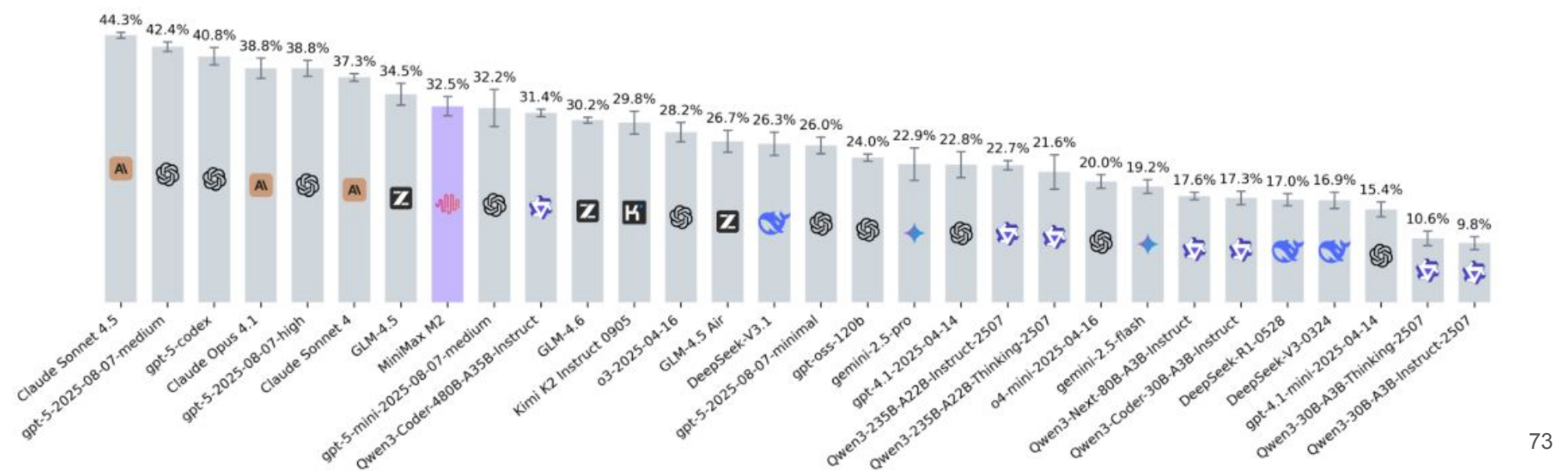
Default

Compact View

Model	266 / 266	Overall	Hard Prompts	Coding	Math	Creative Writing	Instruction Following	Longer Query	Multi-Turn
 grok-4.1		1	1	1	1	1	3	3	1
 grok-4.1-thinking		1	1	1	-	1	1	3	1
 claude-sonnet-4-5-2...		2	1	1	1	1	1	1	1
 gemini-2.5-pro		2	4	8	1	1	3	3	2
 claude-opus-4-1-202...		3	1	1	1	1	1	1	1
 claude-sonnet-4-5-2...		3	1	2	1	1	1	1	1
 gpt-4.5-preview-202...		4	10	9	6	1	4	6	1
 claude-opus-4-1-202...		5	3	2	1	2	1	3	1
 chatgpt-4o-latest-2...		6	8	9	15	5	9	7	1
 gpt-5-high		6	8	8	1	17	9	18	12
 kimi-k2-thinking		6	7	3	1	5	8	7	5
 o3-2025-04-16		7	10	13	1	16	15	23	13
 qwen3-max-preview		7	6	5	1	10	7	6	5
 glm-4.6		10	8	8	1	5	8	7	13
 gpt-5-chat		12	8	10	5	11	9	7	5
 qwen3-max-2025-09-23		12	8	7	1	8	8	7	3
 claude-opus-4-20250...		13	8	3	4	3	3	3	8
 deepseek-r1-0528		13	15	8	10	11	24	22	18
 deepseek-v3.1-termi...		13	17	17	4	3	12	8	18
 deepseek-v3.1-termi...		13	8	8	1	10	8	3	8
 deepseek-v3.2-exp-t...		13	9	7	1	8	8	7	12
 ernie-5.0-preview-1...		13	15	20	1	2	10	7	12

Rank	Model	Resolved Rate (%)	Resolved Rate SEM ($\pm$)	Pass@5 (%)	Cost per Problem (\$)	Tokens per Problem
1	 Claude Sonnet 4.5	44.3%	0.48%	56.9%	\$0.98	2 002 524
2	 gpt-5-2025-08-07-medium	42.4%	0.78%	49.0%	\$0.66	1 460 199
3	 gpt-5-codex	40.8%	1.44%	47.1%	\$0.51	1 722 212
4	 Claude Opus 4.1	38.8%	1.69%	49.0%	\$4.41	1 849 354
5	 gpt-5-2025-08-07-high	38.8%	1.30%	47.1%	\$1.00	1 968 033
6	 Claude Sonnet 4	37.3%	0.62%	47.1%	\$0.96	2 024 614
7	 GLM-4.5	34.5%	1.82%	47.1%	\$1.03	1 704 653
8	 MiniMax M2	32.5%	1.59%	51.0%	\$0.44	1 680 096
9	 gpt-5-mini-2025-08-07-medium	32.2%	3.08%	45.1%	\$0.28	1 152 682
10	 Qwen3-Coder-480B-A35B-Instruct	31.4%	0.62%	41.2%	\$0.61	1 492 492

- Claude Sonnet 4.5 delivers the strongest *pass@1* and *pass@5* results, and it uses more tokens per problem than gpt-5-medium and gpt-5-high despite not running in reasoning mode. This indicates that Sonnet 4.5 adapts its reasoning depth internally and uses its capacity efficiently.
- GPT-5 variants differ in how often they invoke reasoning: gpt-5-medium uses it in ~58% of steps (avg. 714 tokens), while gpt-5-high increases this to ~62% (avg. 1053 tokens). However, this additional reasoning does not translate into better task-solving ability in our setup: gpt-5-medium achieves a *pass@5* of 49.0%, compared to 47.1% for gpt-5-high.
- MiniMax M2 is the most cost-efficient open-source model among the top performers. Its pricing is \$0.255 / \$1.02 per 1M input/output tokens, whereas gpt-5-codex costs \$1.25 / \$10.00 – with cached input available at just \$0.125. In agentic workflows, where large trajectory prefixes are reused, this cache advantage can make models with cheap cache reads more beneficial even if their raw input/output prices are higher. In case of gpt-5-codex, it has approximately the same *Cost per Problem* as MiniMax M2 (\$0.51 vs \$0.44), being yet much more powerful.
- GLM-4.6 reaches the agent's maximum step limit (80 steps in our setup) roughly twice as often as GLM-4.5. This suggests its performance may be constrained by the step budget, and increasing the limit could potentially improve its resolved rate.



SWE-bench

vals.ai

11/14/2025

Solving production software engineering tasks

Scatter Plot

Bar Chart

Table

All Models

Task type : Overall

Models (28) ^		Accuracy ^	Cost / Test ^	Latency ^
1	 GPT 5.1 Codex ⓘ ⚡	70.4 %	\$0.31	1824.70 s
2	 Claude Sonnet 4.5 (Thinking) ⓘ	69.8 %	\$8.26	762.98 s
3	 GPT 5 Codex ⓘ ⚡	69.4 %	\$2.21	405.30 s
4	 Claude Haiku 4.5 (Thinking) ⓘ 💰	68.8 %	\$2.55	1389.23 s
5	 GPT 5 ⓘ	68.8 %	\$1.41	2328.15 s
6	 GPT 5.1 ⓘ	67.2 %	\$0.36	563.34 s
7	 Claude Sonnet 4 (Nonthinking)	65.0 %	\$1.24	426.52 s
8	 Qwen 3 Max	62.4 %	\$1.66	2803.83 s
9	 Grok 4 ⓘ	58.6 %	\$1.21	704.78 s
10	 Grok Code Fast ⓘ	57.6 %	\$1.96	264.68 s